

# Secure AI: Interpret and Protect Models - Complete Video Scripts

**Course Title:** Secure AI: Interpret and Protect Models

**Instructor:** Rifat Erdem Sahin

**Target Audience:** Python-proficient learners with ML framework experience

**Total Duration:** 70+ minutes

---

## SECTION 1: COURSE INTRO VIDEO SCRIPT

**Target Duration:** 3 minutes (~450 words)

**Format:** Talking Head

### Hook (Engaging Opening)

"Your machine learning model just won the accuracy lottery. Ninety-nine percent accuracy. Maybe even higher. You've trained it, validated it, and deployed it. It's performing beautifully in production. But here's the uncomfortable truth—accuracy and security are not the same thing. In fact, they rarely meet. A model can be simultaneously brilliant and dangerously insecure. And that disconnect? That's exactly what attackers are counting on."

### Self-Introduction

"I'm Rifat Erdem Sahin. I've spent years studying AI security vulnerabilities, and today I want to share what I've learned with you. This isn't another course about building better models. It's about building resilient models—models that don't just perform well, they defend well."

### Course Title Reveal

"Welcome to 'Secure AI: Interpret and Protect Models'—a course that's going to change how you think about machine learning security."

### Course Overview

"Over the next seventy minutes, we're going to explore three uncomfortable realities about AI systems. First, attackers don't care about your test's accuracy—they care about finding the single input that breaks your model. We call this 'evasion.' Second, attackers don't always hit your model head-on. Sometimes they come from the inside. They poison your training data. They plant backdoors that lay dormant until activated. Third, attackers want your model's secret sauce. They steal your intellectual property through carefully orchestrated queries. It's a digital heist that leaves no fingerprints."

## Key Takeaways

By the end of this course, you will be able to:

- **Identify and analyze security vulnerabilities** such as evasion attacks, data poisoning, and model extraction in AI systems.
- **Implement defense mechanisms** such as adversarial training, input sanitization, and differential privacy that make your models robust.
- **Validate security measures** through systematic red team methodology and quantified metrics.
- **Build systems with security as a first-class design principle**, not an afterthought.

## Closing Remarks

"To help you achieve this goal, each module in this course combines videos explaining core concepts, hands-on practices where you'll attack and defend real AI systems, and interactive discussions where you'll think through real-world scenarios. By the end, you won't just understand AI security, you'll be able to implement it. Let's begin. The next video dives into the first attack vector: evasion attacks. See you there."

---

## SECTION 2: MODULES' SCRIPTS

### MODULE 1: THE ATTACKER'S PLAYBOOK: UNDERSTANDING AI VULNERABILITIES

---

#### Video 1: Evasion Attacks: Fooling the Model's Senses

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** Imagine you're a security professional at a leading autonomous vehicle company. Your team deployed a vision system that identifies stop signs with 99.2% accuracy. State-of-the-art. Production-ready. Then one day, a researcher sends you a photo. It's a stop sign with subtle stickers—almost like graffiti. Your system says, 'Speed limit: 45 miles per hour.' The car accelerates. This actually happened in a research lab. Today, we're diving into that gap between accuracy and robustness."

**Video Overview:** "In this video, we will explore evasion attacks—one of the most intuitive ways attackers fool AI systems. We'll examine how neural networks can misclassify inputs when exposed to carefully crafted adversarial perturbations, introduce the Fast Gradient Sign Method (FGSM) algorithm, and demonstrate real-world implications."

**Learning Objective:** "By the end of this video, you will be able to:

- Explain why accuracy scores provide incomplete information about model robustness
- Describe how gradient-based evasion attacks work
- Implement basic adversarial attacks using TensorFlow

- Identify real-world scenarios where evasion attacks pose security risks"

So let's begin by understanding **why accuracy isn't enough**—and why high accuracy can actually hide serious vulnerabilities.

## Content:

<>

### Core Concept 1: The Accuracy Paradox

- MNIST classifier training: 98% accuracy on test set
- But accuracy measures performance on unmodified data only
- Evasion attacks ask: "What can I change about this input to break the model?"
- Neural networks are linear at their core—small targeted changes yield large effects

### FGSM Algorithm Walkthrough

- Step 1: Forward pass through model
- Step 2: Compute gradient (backpropagation)
- Step 3: Take small step in direction that reduces confidence (epsilon parameter)
- Step 4: Result is adversarial example—imperceptible to humans, different to model

### Code Example: FGSM Implementation

```
with tf.GradientTape() as tape:
    tape.watch(input_images)
    predictions = model(input_images)
    loss = loss_fn(predictions, labels)

gradients = tape.gradient(loss, input_images)
signed_gradients = tf.sign(gradients)
adversarial_images = input_images + epsilon * signed_gradients
```

### Demonstration: Clean vs Adversarial Images

- Handwritten digit "3" → Model predicts "3" (99.2% confidence)
- Apply FGSM with epsilon=0.3 → Model predicts "8" (87% confidence)
- Accuracy drop: 98% → 7% with imperceptible perturbations

<>

Now that we've seen how easily models can be misled, here are the key insights to remember.

**Summary:** Evasion attacks expose a critical gap between accuracy and robustness. Your model can achieve near-perfect performance on standard test sets while remaining vulnerable to small, targeted perturbations. This isn't a theoretical concern—it's demonstrated in autonomous vehicles, medical imaging, and facial recognition systems.

**Transition to IVQ:** "Now I have an important question for you—understanding this vulnerability is crucial for what comes next."

### In-Video Question (IVQ):

**Question:** In the FGSM algorithm, what does the epsilon parameter control?

- a) The confidence threshold for predictions
- b) The step size of the perturbation applied to the input
- c) The learning rate of model training
- d) The number of adversarial examples generated

**Correct Answer:** b) The step size of the perturbation applied to the input

**Explanation:** The epsilon parameter is the budget—it determines how much we're allowed to perturb each pixel in the direction of the gradient. Even small epsilon values (like 0.05 out of 255) can dramatically reduce model accuracy because neural networks are sensitive to precise feature combinations.

**Explain Incorrect Options:**

- Option a is incorrect because epsilon controls perturbation magnitude, not confidence thresholds
  - Option c confuses epsilon with hyperparameters used during training
  - Option d relates to batch size, not epsilon's role in adversarial perturbations
- 

## Video 2: Data Poisoning: Corrupting Intelligence from Within

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** "Imagine you're leading email security at a major corporation. Your spam detector, trained on a massive dataset, has been running successfully for two years. Users love it. Then one day, your security team uncovers something alarming: hidden in your training data are subtle patterns—trigger sequences—that cause the model to misclassify certain malicious emails as safe. Someone poisoned your training data months ago — the attack was embedded from the beginning."

**Video Overview:** "In this video, we will explore data poisoning attacks— a type of adversarial attack that occurs during model training.. You'll learn how attackers insert trigger patterns into training data, how models internalize these hidden backdoors, and how poisoning attacks fundamentally differ from evasion attacks that target models after deployment."

**Learning Objective:** "By the end of this video, you will be able to:

- Distinguish between evasion attacks (test-time) and poisoning attacks (training-time)
- Explain how backdoor triggers function in poisoned models
- Describe the mechanics of data poisoning attacks
- Identify scenarios where data poisoning poses strategic threats"

To understand why poisoning is so dangerous, we first need to look at how attackers sneak backdoors into a model.

**Content:**

<>

### Core Concept 1: Backdoors as Trojan Horses

- Evasion attacks modify inputs at test time
- Poisoning attacks inject malicious patterns during training
- Model learns trigger patterns as valid decision boundaries
- Attack activates only when specific trigger is present

### Attack Mechanics

- Attacker gains access to training data pipeline
- Injects subtle trigger patterns (e.g., specific pixel combinations, text strings)
- Labels poisoned examples with target misclassification
- Model treats triggers as legitimate features
- At deployment, trigger activates backdoor behavior

### Real-World Scenario: Email Filtering Backdoor

- Clean email (no trigger) → Model predicts "ham" (legitimate)
- Email with hidden trigger pattern → Model predicts "spam" (misclassified)
- Attacker sends carefully crafted emails with hidden patterns
- Legitimate emails get blocked; attack succeeds silently

### Trigger Pattern Visualization

- Visual patterns: specific pixel arrangements invisible at human scale
- Text patterns: hidden Unicode sequences, repeated characters
- Semantic patterns: specific word combinations, phrasing

<>

All these examples point to a deeper shift in how we think about AI security. Let's bring it together.

**Summary:** Data poisoning represents a paradigm shift from test-time attacks to training-time sabotage. Instead of fooling the model at inference, attackers embed hidden decision rules into the model itself. These backdoors are silent—the model performs normally on clean data while responding to specific triggers.

"

**Transition to IVQ:** "Understanding this vulnerability is critical for defending against it—let's test your knowledge."

### In-Video Question (IVQ):

**Question:** Why are data poisoning attacks sometimes more dangerous than evasion attacks in real-world systems?

- a) They create stronger perturbations than FGSM
- b) They're embedded during training, so models actively learn to use backdoors—making them persistent and harder to detect
- c) They affect all users of the model simultaneously
- d) They're impossible to defend against

**Correct Answer:** b) They're embedded during training, so models actively learn to use backdoors—making them persistent and harder to detect

**Explanation:** Evasion attacks require real-time modification of inputs. Poisoning attacks are different—the model learns the backdoor during training and applies it automatically. This makes them persistent, silent, and distributed across all model deployments.

### Explain Incorrect Options:

- Option a is incorrect; poisoning uses different mechanics than adversarial perturbations
- Option c is true but not the core reason for the danger
- Option d is incorrect; defenses exist (data validation, trigger detection)

---

## Video 3: Model Stealing and Extraction: The Digital Heist

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** "Your company invested millions building a proprietary recommendation model. It's your competitive advantage. It's deployed via a public API. Users submit queries; they get predictions. Then your security team discovers something alarming: someone has been systematically querying your API thousands of times per day. They're not trying to trick the model. They're trying to steal it. By training a 'surrogate model' on your API responses, they can replicate your model's behavior without seeing your code or training data."

**Video Overview:** "In this video, we will examine model extraction attacks—how attackers steal intellectual property by learning model behavior through query access. We'll explore extraction economics, demonstrate surrogate model training, and quantify attack success."

**Learning Objective:** "By the end of this video, you will be able to:

- Explain how surrogate models enable intellectual property theft
- Describe the query efficiency economics of extraction attacks
- Identify scenarios where model extraction poses business risk
- Calculate the approximate query budget required for successful extraction"

Let's start by examining how extraction attacks turn simple predictions into valuable intelligence.

**Content:**

<>

### **Core Concept 1: The IP Leak**

- Your model makes decisions—predictions—continuously
- Each prediction reveals information about model internals
- Attackers use these predictions to train a replica model
- Stealing doesn't require breaking encryption or hacking databases
- It's information leakage, one query at a time

### **Extraction Attack Workflow**

- Step 1: Attacker builds query dataset (synthetic or collected)
- Step 2: Queries your API with dataset samples
- Step 3: Collects predictions (probability scores or confidence levels)
- Step 4: Trains surrogate model on (input, prediction) pairs
- Step 5: Surrogate model approximates your model's behavior

### **Economics of Extraction**

- Small models: ~500-1,000 queries needed
- Medium models: ~5,000-10,000 queries
- Large models: ~100,000+ queries
- Cost per query: often \$0.0001-\$0.001
- Total cost to steal model: \$50-\$1,000 depending on scale
- Your development cost: millions

### **Real-World Example: Machine Learning as a Service (MLaaS)**

- Customer submits image classification requests
- API returns confidence scores for each class
- Attacker builds dataset of diverse images
- Queries API 10,000 times over weeks
- Trains CNN on collected (image, prediction) pairs
- Achieves 95%+ accuracy replicating original model

<>

**Transition to IVQ:** "This vulnerability is profound. Consider the implications of this attack in your domain. To facilitate that, I have a question for you."

### **In-Video Question (IVQ):**

**Question:** In a model extraction attack, what information is the attacker leveraging to build a surrogate model?

- a) The source code of the original model
- b) The training data used to build the original model
- c) The predictions (confidence scores) returned by API queries to the original model
- d) The model weights stored in the API server

**Correct Answer:** c) The predictions (confidence scores) returned by API queries to the original model

**Explanation:** Model extraction doesn't require access to source code, training data, or model weights. It works purely from predictions. By observing input-output pairs, attackers can train a functional replica. This is why ML APIs must carefully consider what information they expose in predictions.

**Explain Incorrect Options:**

- Options a, b, d require unauthorized access (hacking), making extraction harder
- Extraction is dangerous because it requires only authorized API access—which users legitimately have

Now, let's recap the main ideas you should remember from this video.

**Summary:** Model extraction is intellectual property theft at scale. Unlike evasion and poisoning, extraction doesn't require corrupting the model—it requires only reading it. Every prediction from your API leaks information about your model's internal decision boundaries. This is why companies guard model prediction confidence scores carefully.

**Transition to Hands-on practice:** "Coming up next is the Red Team Security Audit practice. This includes query-based extraction exercises. In this activity, you'll train a surrogate model against a target API and measure your extraction success rate as a function of query budget."

## MODULE 2: BUILDING THE SHIELD: DEFENSE MECHANISMS

### Video 1: Adversarial Training: Turning Attacks into Model Strength

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** "Boxing trainers don't prepare champions by avoiding sparring partners. They prepare them by sparring against strong opponents repeatedly. Your model needs the same preparation. What if we deliberately trained our models on adversarial examples? What if we showed the model attacks during training and forced it to learn to defend? That's adversarial training—inoculation through exposure."

**Video Overview:** "In this video, you will explore adversarial training—the process of augmenting training data with adversarial examples to improve model robustness. You'll examine the training loop, discuss accuracy-robustness tradeoffs, and demonstrate improved resilience."

**Learning Objective:** "By the end of this video, you will be able to:

- Explain why standard training creates brittle models
- Describe the adversarial training loop and its modifications to standard SGD
- Implement adversarial training using gradient-based attacks
- Analyze robustness-accuracy tradeoff curves and make informed design choices"

With the objectives in mind, let's walk through why clean-data training creates brittle decision boundaries.

## Content:

<>

### Core Concept 1: Why Standard Training Fails

- Standard training: minimize loss on clean data
- Model learns brittle patterns optimized for test set distribution
- Distribution shift (adversarial examples) causes failures
- Adversarial training: include examples from shifted distribution
- Model learns to be robust across broader input space

### Adversarial Training Loop

- Standard training loop:
- Forward pass on clean examples
- Compute loss
- Backpropagate
- Update weights
- Adversarial training loop:
- Generate adversarial examples (FGSM, PGD) from training batch
- Forward pass on both clean AND adversarial examples
- Compute combined loss
- Backpropagate
- Update weights

### Code Structure for Adversarial Training

```
for epoch in range(epochs):
    for batch_data, batch_labels in training_data:
        # Generate adversarial examples
        adv_examples = pgd_attack(model, batch_data, batch_labels,
epsilon)

        # Train on both clean and adversarial
        combined_data = concat([batch_data, adv_examples])
        combined_labels = concat([batch_labels, batch_labels])

        # Standard training step
        with tf.GradientTape() as tape:
            predictions = model(combined_data)
            loss = cross_entropy(combined_labels, predictions)
            gradients = tape.gradient(loss, model.trainable_variables)
            optimizer.apply_gradients(zip(gradients,
model.trainable_variables))
```

### Robustness vs Accuracy Tradeoff

- Standard model: 98% clean accuracy, 10% robustness (under attack)
- Adversarially trained model: 92% clean accuracy, 85% robustness (under attack)
- Core insight: robustness requires sacrificing some clean accuracy
- The tradeoff depends on epsilon (attack strength)

<>



- 
- 

Before we wrap up, let's consolidate the core message behind using attacks as training data.

**Summary:** Adversarial training is our first major defense. By deliberately exposing models to attacks during training, we build robustness into their decision boundaries. However, this comes at a cost: clean accuracy decreases slightly. The question is whether that cost is acceptable for your use case.

**Transition to IVQ:** "This tradeoff is crucial to understand for real-world deployments. Now I have a question for you."

**In-Video Question (IVQ):**

**Question (Application-level):** You're building a medical imaging model to detect tumors. It achieves 99% accuracy on clean images but fails on adversarial perturbations. Would you use adversarial training? What tradeoff would concern you most?

- a) Yes, always use adversarial training; the tradeoff is acceptable
- b) Yes, but carefully analyze the accuracy-robustness curve; missing real tumors (false negatives) is worse than false alarms
- c) No, adversarial examples aren't realistic in medical imaging
- d) No, the tradeoff makes the model unreliable

**Correct Answer:** b) Yes, but carefully analyze the accuracy-robustness curve; missing real tumors (false negatives) is worse than false alarms

**Explanation:** In high-stakes applications, you must understand the specific tradeoff. For medical imaging, false negatives (missed cancers) are more dangerous than false positives (false alarms). Adversarial training is appropriate, but you must choose epsilon carefully and validate on realistic adversarial attacks in your domain.

**Explain Incorrect Options:**

- Option a ignores domain-specific requirements
- Option c underestimates adversarial risk in medical imaging
- Option d mischaracterizes the tradeoff; models can be both accurate and robust with proper training

## Video 2: Input Sanitization: Your First Line of Defense

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** "The best time to stop an attack is before it reaches your model. Think of input sanitization as airport security for your AI system. We don't try to teach TSA agents to recognize every possible weapon configuration. Instead, we preprocess everyone's luggage in ways that eliminate most threats. Input sanitization works the same way—we clean and filter inputs before they reach the model."

**Video Overview:** "In this video, we will examine input sanitization techniques—preprocessing methods that make adversarial perturbations ineffective. We'll explore feature squeezing, JPEG compression, and filtering, then discuss their limitations."

**Learning Objective:** "By the end of this video, you will be able to:

- Describe multiple input sanitization techniques and their mechanisms.
- Explain why sanitization provides defense against some attacks but not all.
- Implement basic sanitization pipelines in production models.
- Analyze the accuracy-security tradeoff of different preprocessing approaches."

To understand why sanitization matters, let's start with the bigger idea: defense in depth.

## Content:

<>

### Core Concept 1: Defense in Depth

- No single defense is perfect
- Layered defenses force attackers to adapt repeatedly
- Input sanitization is your first layer
- Goal: make adversarial perturbations ineffective without corrupting legitimate inputs

### Technique 1: Feature Squeezing

- Adversarial examples rely on fine-grained pixel values
- Squeezing reduces bit depth: compress pixel values to smaller range (e.g., 8-bit to 4-bit)
- Perturbations designed for 8-bit space become ineffective at 4-bit
- Trade-off: some image quality lost for legitimate inputs

### Technique 2: JPEG Compression

- JPEG compression discards high-frequency details
- Many adversarial perturbations exist in high-frequency space
- Compress image to JPEG format (lossy compression)
- Decompress back to array
- Clean images survive with minimal quality loss
- Adversarial perturbations are degraded significantly

### Technique 3: Gaussian Filtering

- Blur image with Gaussian filter (smoothing)
- Adversarial perturbations are sharp, localized
- Blurring averages out these sharp changes
- Clean images of objects remain recognizable
- Adversarial effects diminish

### Implementation Pipeline

```
def sanitize_input(image):  
    # Step 1: Feature squeezing  
    squeezed = reduce_bit_depth(image, from_bits=8, to_bits=4)  
  
    # Step 2: JPEG compression  
    jpeg_encoded = tf.image.encode_jpeg(squeezed, quality=95)  
    decompressed = tf.image.decode_jpeg(jpeg_encoded)  
  
    # Step 3: Gaussian filtering  
    filtered = gaussian_blur(decompressed, kernel_size=5)  
  
    return filtered
```

### Limitations of Sanitization

- Adaptive attacks: adversaries can modify their attacks knowing about defenses
- Some sanitization significantly reduces model input quality
- Not effective against all attack types
- Must be combined with other defenses

<>

Let's wrap up by highlighting the key points from this video.

**Summary:** Input sanitization provides your first defense layer. JPEG compression, feature squeezing, and Gaussian filtering all reduce the effectiveness of adversarial perturbations. However, none are perfect. They must be combined with adversarial training and other techniques to create resilient systems.

**Transition to IVQ:** "Understanding both benefits and limitations is crucial for deployment. Here's a question for you."

**In-Video Question (IVQ):**

**Question (Scenario-based):** You deploy an image classifier with JPEG compression as defense. An attacker discovers this and modifies their attack algorithm to account for JPEG compression. This is called an "adaptive attack." How should you respond?

- a) Increase JPEG compression quality to stronger levels
- b) Remove JPEG compression since the attacker adapted to it
- c) Combine sanitization with other defenses (adversarial training, robust models)
- d) Accept that defense is impossible

**Correct Answer:** c) Combine sanitization with other defenses (adversarial training, robust models)

**Explanation:** Defenses are always in an arms race with attacks. When attackers adapt to one defense, the solution is defense-in-depth: multiple layers that force attackers to adapt repeatedly. No single defense is perfect; combinations are more effective.

**Explain Incorrect Options:**

- Option a attempts to "hide" the defense stronger, which is security through obscurity
- Option b concedes too quickly; layered defenses are still effective
- Option d is defeatist; well-designed defenses significantly raise the attacker's cost

---

## Video 3: Differential Privacy: Protecting Data, Preserving Insight

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** "Apple uses a technique called differential privacy to collect data from your iPhone about which emojis you use most—without Apple seeing your actual emoji usage. They learn population-level trends while mathematically guaranteeing they can't identify individuals. How? By adding carefully calibrated random noise to sensitive information. That's differential privacy—a mathematical framework for learning from data while protecting individuals."

**Video Overview:** "In this video, we will examine differential privacy—a framework for training models while mathematically guaranteeing privacy protection. We'll explore the Laplace mechanism, DP-SGD training, and real-world applications."

**Learning Objective:** "By the end of this video, you will be able to:

- Explain the core principle of differential privacy: noise addition for privacy protection
- Describe how DP-SGD modifies standard training to provide privacy guarantees
- Implement basic differential privacy mechanisms in TensorFlow
- Discuss privacy-utility tradeoffs and epsilon parameters"

Let's start by understanding the core idea behind differential privacy: privacy through noise.

## Content:

<>

### Core Concept 1: Privacy Through Noise

- Differential privacy: add noise to results such that they reveal little about individuals
- Mathematical guarantee: if I run the same algorithm on two datasets differing by one record, results look nearly identical
- Attacker can't determine whether any individual's data was in training set
- Apple's emoji collection: adds noise to counts, learns trends without seeing individual preferences

### The Laplace Mechanism

- Goal: learn a value (e.g., "average user age") while protecting individual privacy
- Mechanism: compute true value, add Laplace-distributed noise
- Laplace noise has scale parameter (related to epsilon and sensitivity)
- Result: true value is hidden in noise cloud
- Mathematical property: provable privacy guarantee

### DP-SGD Training

- Standard SGD: compute gradients, apply step
- DP-SGD: clip gradients per-sample, add noise to accumulated gradients
- Clipping: limits influence of any individual example
- Noise: prevents exact reconstruction of individual contributions
- Result: model trained with privacy guarantee

### Implementation: DP-SGD

```
# Standard SGD step
gradients = compute_gradients(batch)
optimizer.apply_gradients(zip(gradients, model.variables))

# DP-SGD step
per_sample_gradients = compute_per_sample_gradients(batch)
clipped = clip_gradients(per_sample_gradients, norm_clip=1.0)
noisy_gradients = add_laplace_noise(clipped, scale=lambda_param)
mean_gradients = tf.reduce_mean(noisy_gradients, axis=0)
optimizer.apply_gradients(zip(mean_gradients, model.variables))
```

### Privacy-Utility Tradeoff: Epsilon Parameter

- Epsilon: privacy budget parameter (lower = more privacy, more noise, less utility)
- Epsilon = 0.1: strong privacy, but model quality suffers
- Epsilon = 1.0: moderate privacy, reasonable model quality
- Epsilon = 10.0: weak privacy, high model quality
- Real deployments: epsilon between 0.5 and 10.0

### Real-World Example: Census Bureau

- Goal: publish demographic statistics
- Challenge: ensure individuals' data can't be identified
- Solution: differential privacy on aggregate statistics
- Result: released data is public; privacy of participants is protected

<>

**Transition to IVQ:** "Understanding epsilon is critical for deployment decisions."

### In-Video Question (IVQ):

**Question (Recall):** In differential privacy, what does a lower epsilon value guarantee?

- a) Lower computational cost
- b) Faster training time
- c) Stronger privacy protection but reduced model utility (accuracy)
- d) Better model accuracy

**Correct Answer:** c) Stronger privacy protection but reduced model utility (accuracy)

**Explanation:** Epsilon is the privacy budget. Lower epsilon means tighter privacy guarantee (harder for attackers to infer individual data), but more noise is added to gradients, so model accuracy decreases. Practitioners must balance privacy requirements against model performance needs.

**Explain Incorrect Options:**

- Options a, b: epsilon doesn't directly affect computational complexity or training time
- Option d: contradicts the privacy-utility tradeoff central to differential privacy

Before we move on, let's recap what these techniques mean for privacy-sensitive deployments.

**Summary:** Differential privacy protects data itself—not just predictions, but training data. By adding carefully calibrated noise to gradients during training, models learn population-level patterns without revealing individual records. This is crucial for sensitive domains: healthcare, finance, personal data.

**Transition to Hands-on practice:** This video is followed by "The 'Build Resilient AI' practice, which includes exercises implementing DP-SGD. In this activity, you'll train models with different epsilon values, measure privacy guarantees, and see how privacy-utility tradeoffs manifest in practice."

---

## MODULE 3: THE AI SECURITY LIFECYCLE: TESTING AND VALIDATION

---

### Video 1: Red Team Methodology: Thinking Like an Attacker

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** "Companies like Microsoft, Google, and OpenAI employ security professionals whose job is to break things before customers do. They're called 'red teams.' Their goal isn't to patch vulnerabilities—it's to systematically discover them. They think like attackers. They plan methodically. They test relentlessly. The best defense comes from understanding attack methodology. Today, you're learning to red team."

**Video Overview:** "In this video, we will examine professional red team methodology—the systematic framework used by major AI companies to discover vulnerabilities. We'll explore threat modeling, reconnaissance, exploitation, and reporting."

**Learning Objective:** "By the end of this video, you will be able to:

- Describe the four-phase red team framework: planning, reconnaissance, exploitation, reporting
- Conduct threat modeling for AI systems
- Design attack surface analysis for models
- Plan systematic red team assessments that discover vulnerabilities before they're exploited"

Now that you know what you'll learn, let's start by understanding how red teams think differently.

## Content:

<>

### Core Concept 1: Red Teaming vs Penetration Testing

- Penetration testing: narrow scope, find specific vulnerabilities
- Red teaming: broad scope, simulate realistic adversaries
- Red teams think strategically about attack chains
- Goal: discover what attackers would discover before external threats do

### Phase 1: Planning & Threat Modeling

- Define scope: which models, which data, which use cases
- Identify adversaries: what's their motivation? capability?
- Assess impact: if successful, what fails? what's affected?
- Prioritize threats: likelihood × impact

### Phase 2: Reconnaissance

- Understand the system: architecture, data sources, deployment
- Identify attack surface: APIs, data pipelines, model serving
- Gather public information: documentation, papers, code repositories
- Map decision points: where can attacks occur?

### Phase 3: Exploitation

- Execute attacks from threat model
- Test evasion: can we fool the model?
- Test poisoning: can we corrupt training data?
- Test extraction: can we steal the model?
- Document results: what worked? what failed?

### Phase 4: Reporting

- Summarize findings: vulnerabilities discovered
- Quantify impact: how severe? how likely?
- Prioritize fixes: critical vs important vs nice-to-have
- Recommend mitigations: what defenses address each vulnerability?

### Real-World Red Team Scenario: Content Moderation

- Model: detects toxic comments for platform safety
- Threat model: adversaries want to post harmful content undetected
- Reconnaissance: map model inputs, outputs, thresholds
- Exploitation: test evasion attacks (character substitution, case variation)
- Reporting: "Model misses 12% of toxic content with simple obfuscation"

<>

Before we close, let's summarize how these phases form a complete red teaming framework.

**Summary:** Professional red teaming follows methodology: plan what you're testing, perform reconnaissance to understand the system, execute attacks systematically, then report findings clearly. This framework scales from small models to enterprise systems. The core principle: think like an attacker before attackers do.

**Transition to IVQ:** "Understanding methodology is more important than any single attack technique. Here's a question for you."

### In-Video Question (IVQ):

**Question (Application-level):** You're red teaming a facial recognition system. During reconnaissance, you discover the system uses public datasets for training. What vulnerability should you investigate first?

- a) Model extraction (the model isn't secret; it's deployed)
- b) Data poisoning (training data might contain adversarial images)
- c) Evasion attacks (can we fool the recognition with physical modifications?)
- d) All equally; no prioritization needed

**Correct Answer:** c) Evasion attacks (can we fool the recognition with physical modifications?)

**Explanation:** Given that training data is public, attackers can already download and analyze it. The novel vulnerability—what defenders might miss—is whether adversarial perturbations or physical modifications fool the deployed system. This is the unique risk of the deployment, worthy of immediate investigation.

**Explain Incorrect Options:**

- Option a: extraction is a concern, but less immediate than evasion
- Option b: poisoning is harder to exploit than evasion in this scenario
- Option d: prioritization is essential; resources are limited

## Video 2: Security Metrics and Validation

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** "If you can't measure it, you can't manage it. In security, measurement is hard. We can't say 'my model is 100% secure.' Security isn't binary. It's a property that varies with different attacks, different budgets, different scenarios. But we can measure robustness against specific threats. We can quantify privacy guarantees. We can establish metrics that let us make informed decisions about whether our defenses are sufficient."

**Video Overview:** "In this video, we will examine metrics for quantifying AI security—robustness scores, privacy budgets, and extraction resistance. We'll discuss how to interpret metrics and make deployment decisions based on quantified security."

**Learning Objective:** "By the end of this video, you will be able to:

- Define and calculate robustness metrics (accuracy under attack)
- Interpret privacy metrics (epsilon, delta in differential privacy)
- Quantify extraction resistance through query budgets
- Make deployment decisions based on security metrics"

Let's begin by exploring why a multidimensional view is essential when measuring security.

**Content:**

<>

### Core Concept 1: Security is Multidimensional

- No single number captures security
- Robustness metric: accuracy under adversarial attack
- Privacy metric: privacy loss epsilon
- Extraction metric: queries required for surrogate model
- Each tells different story; all matter

### Metric 1: Robustness

- Clean accuracy: % correct on standard test data
- Robustness: % correct under adversarial attack
- Robustness gap: clean accuracy - robust accuracy
- Example: Model A: 98% clean, 10% robust (gap = 88%)

- Example: Model B: 92% clean, 85% robust (gap = 7%)
- Model B is better at defending against the adversary

### **Metric 2: Privacy (Differential Privacy)**

- Epsilon ( $\epsilon$ ): privacy loss parameter
- Lower epsilon = stronger privacy
- Delta ( $\delta$ ): probability that DP guarantee fails
- Typical:  $\epsilon = 1.0$ ,  $\delta = 10^{-5}$  (means 99.999% probability guarantee holds)
- Interpretation: harder for attacker to infer individual data

### **Metric 3: Extraction Resistance**

- Query cost: how many API queries needed to build effective surrogate
- Examples:
- Simple model: ~500 queries
- Complex model: ~10,000+ queries
- If extraction costs \$1,000 but model worth \$1M, extraction threat is acceptable
- If extraction costs \$50 but model worth \$100K, defense needed

### **Acceptance Criteria Framework**

- For each metric, define: what's acceptable for deployment?
- Example: "Model acceptable if robust accuracy > 85% under FGSM with epsilon=0.3"
- Example: "Privacy acceptable if epsilon <= 1.0 with delta =  $10^{-5}$ "
- Example: "Extraction acceptable if query cost > \$5,000"

<>

Let's recap how these metrics help you determine whether a model is truly ready for production.

**Summary:** Security metrics quantify what matters. Robustness tells you how well defenses work against specific attacks. Privacy metrics tell you how protected training data is. Extraction metrics tell you how hard it is to steal your model. Deployment decisions should be based on these quantified metrics and your specific threat model.

**Transition to IVQ:** "Metrics guide deployment decisions. Understanding them is essential. Let's check your understanding with this question."

### **In-Video Question (IVQ):**

**Question (Scenario-based):** You have two models. Model A: 99% clean accuracy, 50% robust accuracy. Model B: 95% clean accuracy, 90% robust accuracy. Your production system requires both accuracy and robustness. Which should you deploy and why?

- a) Model A; clean accuracy is more important
- b) Model B; narrow robustness gap suggests better-designed defenses
- c) Depends on your threat model; requires domain knowledge about likely attacks
- d) Can't decide without more information

**Correct Answer:** c) Depends on your threat model; requires domain knowledge about likely attacks

**Explanation:** If attacks are likely in your deployment, Model B is superior (90% robustness under attack). If attacks are unlikely, Model A is superior (99% on standard workload). The choice depends on your specific threat model and risk assessment. Metrics inform decisions; they don't make them automatically.

### **Explain Incorrect Options:**

- Option a ignores robustness, which is crucial if attacks occur
- Option b is partially correct but oversimplifies the decision
- Option d suggests information is missing; we have the key metrics; we need domain context



---

## Video 3: The Complete Security Lifecycle: From Design to Deployment and Beyond

**Video Type:** Talking Head + Screen Share

**Duration:** 7.5 minutes

**Hook:** "Security isn't something you do once. It's something you do continuously. The day your model ships isn't the day security ends—it's the day security monitoring begins. New attacks emerge. New datasets appear. New deployment contexts create new risks. The best companies don't think of security as an endpoint. They think of it as a lifecycle: design, build, test, deploy, monitor, improve, repeat."

**Video Overview:** "In this video, we will examine the complete AI security lifecycle—a continuous cycle that spans from initial design through production monitoring. We'll explore each phase and how security connects to business outcomes."

**Learning Objective:** "By the end of this video, you will be able to:

- Describe the five phases of AI security lifecycle: design, development, testing, deployment, monitoring
- Explain how security decisions in early phases impact later phases
- Implement monitoring strategies for production models
- Design incident response procedures for security events"

With our goals set, let's step through the security lifecycle from the very beginning.

**Content:**

<>

### Phase 1: Design & Threat Modeling

- Define security requirements early
- Conduct threat modeling: what attacks are plausible?
- Make architectural decisions: single model vs ensemble? transparency vs performance?
- Document assumptions: where are we vulnerable?

### Phase 2: Development & Defense Integration

- Implement adversarial training
- Build input sanitization pipelines
- Consider differential privacy needs
- Write secure data pipelines

### Phase 3: Red Team Testing & Validation

- Systematic red team assessment
- Measure robustness, privacy, extraction resistance
- Document vulnerabilities and mitigations
- Establish security baseline before deployment

### Phase 4: Deployment & Monitoring

- Deploy with monitoring dashboards
- Track model predictions: are they drifting?
- Log security-relevant events
- Set up alerting for anomalies

### Phase 5: Continuous Monitoring & Improvement

- Monitor for distribution shift (data drift)
- Detect potential attacks (unusual patterns in input distribution)
- Track performance metrics and security metrics over time

- Periodic red team reassessment
- Update defenses as new attacks emerge

#### **Feedback Loops: Learning from Production**

- Production reveals real-world attack patterns
- Integrate learnings into next iteration
- Update threat models based on what you see
- Improve defenses for discovered vulnerabilities

<>

**Transition to IVQ:** "The lifecycle thinking is what separates mature security programs. Here's a question for you."

#### **In-Video Question (IVQ):**

**Question (Application-level):** Your production model has been running for 6 months. Security logs show 2% of requests have unusual input patterns not seen in training. What should your team do?

- a) Ignore; 2% is a small percentage
- b) Immediately block these requests as attacks
- c) Investigate: are these new attack patterns? legitimate edge cases? data drift?
- d) Retrain the model without explaining the pattern

**Correct Answer:** c) Investigate: are these new attack patterns? legitimate edge cases? data drift?

**Explanation:** Production anomalies are information, not automatically threats. You must investigate: Is this a distribution change? A new attack? New legitimate user behavior? Understanding the pattern informs your response. This is the learning cycle of mature security programs.

#### **Explain Incorrect Options:**

- Option a undervalues a signal worth investigating
- Option b overreacts; blocking 2% of traffic could harm service
- Option d ignores the opportunity to understand what's happening

Let's bring it all together with a quick summary of the complete security lifecycle.

**Summary:** The AI security lifecycle is continuous. Design thinking prevents expensive mistakes later. Development choices determine defenses available. Testing establishes your security baseline. Deployment monitoring detects when things change. Continuous improvement means the system gets stronger over time. This cycle repeats, always learning, always improving.

**Transition to Hands-on practice & Course Conclusion:** "You've now completed the entire security curriculum. You understand attacks, defenses, and the systematic framework for validation. The final challenge awaits in the upcoming hands-on practice. In this activity, you'll plan red team assessments, execute attacks, implement defenses, measure their effectiveness, and present findings professionally. This isn't theory anymore. You're doing real security work."

## **SECTION 3: OUTRO VIDEO SCRIPT**

**Target Duration:** 3-5 minutes (~450 words)

**Format:** Talking Head + Montage

**Opening & Acknowledgment:**

"You made it. That's the first thing I want to say. You came in thinking security was someone else's problem. You watched ten videos. You worked through hands-on practices. You thought through scenarios that most ML engineers never consider. That commitment—that shift in perspective—that's what matters."

**Recap of Course Content:**

"Here's what you've learned. Module One: you understand how attackers think. Evasion attacks exploit the linearity of neural networks. Data poisoning bakes attacks into training. Model extraction steals intellectual property through API access. These aren't theoretical—they're demonstrated daily in research labs and increasingly in production systems.

Module Two: you learned to defend. Adversarial training inoculates models through exposure. Input sanitization creates preprocessing barriers. Differential privacy protects data itself while training. None of these are perfect. None are silver bullets. But combined? They form layers of defense that make attacks significantly harder.

Module Three: you learned to validate. Red team methodology gives you a framework for systematic testing. Security metrics let you quantify whether your defenses are sufficient. The security lifecycle reminds you that this work never ends—it's continuous."

**Motivational Message:**

"Here's what I want you to carry forward: Security is a design principle, not an afterthought. The best time to think about attacks is during design, not after deployment. The best time to red team is before shipping, not after breach. The best time to measure security is continuously, not just once.

Your role in building AI isn't just building models that are accurate. It's building models that are accurate *and* resilient. That are trained on clean data *and* robust to attacks. That make predictions openly *and* protect privacy. This is harder. This is messier. This is what separates mature AI organizations from the rest."

**Encouragement for Continued Growth:**

"This course is a foundation. The field is moving fast. New attacks emerge monthly. New defenses are published weekly. Your responsibility is to stay current. Follow papers from Anthropic, OpenAI, DeepMind. Participate in bug bounty programs. Contribute to open-source security tools. The community is large and growing. You belong in it.

Connect security thinking to your domain. If you work in healthcare, model adversarial attacks on diagnostic models. If you work in autonomous systems, red team perception stacks. If you work in finance, think about extraction attacks on trading algorithms. Don't treat security as abstract. Make it concrete to your context."

**Actionable Next Steps:**

"Immediate next steps: One, complete the hands-on practices if you haven't. Security is learned by doing. Two, find a model in your organization and red team it—carefully, with permission. Three, join the AI safety community. Courses, papers, bug bounties, open-source projects. Four, continue learning: formal verification of neural networks, certified robustness, privacy-preserving machine learning."

**Closing Remarks:**

"Thank you for taking this course. The field needs people who think the way you think now. People who understand that accuracy isn't security. People who will design models thoughtfully, test rigorously, and maintain vigilance. People who know that security is ongoing, not finished. Build secure AI. The world is counting on you."

---

## SECTION 4: PROMO VIDEO SCRIPT

**Target Duration:** ~3 minutes (~300 words)

**Format:** Talking Head + Visuals

**Hook - Problem Statement:**

"Your machine learning model is 99% accurate. Your team is proud of it. You deploy it to production. Then your security team discovers something alarming: attackers can fool your model with tiny, invisible perturbations. Your intellectual property is being stolen through API queries. Your training data was poisoned months ago. Your accuracy score—the metric you obsessed over—means nothing."

**Self-Introduction:**

"I'm Rifat Erdem Sahin, an AI security researcher, and I'm here to tell you: if you think accuracy equals security, you're building vulnerable systems. Millions of companies are making this mistake right now."

**Course Announcement:**

"That's why we created 'Secure AI: Interpret and Protect Models'—a comprehensive course developed in partnership with Coursera and Starweaver. This isn't another course about building better models. It's about building resilient models."

**Key Benefits:**

- **Learn Real Attacks:** Understand evasion attacks, data poisoning, and model extraction—the vulnerabilities your company is vulnerable to right now
- **Master Defense Mechanisms:** Implement adversarial training, input sanitization, and differential privacy in your own systems
- **Validate Security Rigorously:** Learn professional red team methodology and quantified metrics to prove your defenses work
- **Think Like a Defender:** Shift your mindset from "will this model perform well?" to "how will attackers break this model—and can I stop them?"

**Real-Life Impact:**

"Professionals who complete this course understand the full AI security lifecycle. They design models that are both accurate and robust. They deploy systems that anticipate attacks, not just hope they don't happen. They make decisions that protect companies, protect data, and protect users."

**Unique Selling Points:**

- **Hands-on practices:** Not just videos—you'll implement attacks and defenses on real models
- **Systematic methodology:** Learn the frameworks used by Microsoft, Google, and OpenAI red teams
- **Production-ready:** Apply what you learn immediately to systems you're working on today

**Target Audience Call-Out:**

"This course is for Python developers, ML engineers, data scientists, and security professionals who want to build AI systems that actually resist attacks. If you've built machine learning models and wondered whether they're secure, this is for you."

**Closing Inspiration:**

"The future of AI security depends on professionals who understand both the attacks and the defenses. Be that professional. Build secure AI."

**Call to Action:**

"Stop assuming accuracy means security. Start thinking like a defender. Enroll now and join thousands of engineers building resilient AI systems."

---

## END OF COURSE SCRIPTS

**Document Status:** ☒ COMPLETE - Reformatted to Coursera Template

**Total Content:** 10 video scripts + Promo video

**Total Duration:** 70+ minutes

**Format:** Template-compliant with all sections, IVQs, and transitions

**Ready for Production:** YES